SR UNIVERSITY

School of Computer Science and Artificial Intelligence

Lab Assignment # 8

Name of Student : K.Saikiran
Enrollment No. : 2303A51539
Batch No. : 22

Task Description #1 (Username Validator – Apply AI in Authentication Context)
• Task: Use AI to generate at least 3 assert test cases for a function is_valid_username and then implement the function using Test-Driven Development principles.
• Requirements:
o Username length must be between 5 and 15 characters.
o Must contain only alphabets and digits.
o Must not start with a digit.
o No spaces allowed.
Example Assert Test Cases:
assert is_valid_username("User123") == True
assert is_valid_username("12User") == False
assert is_valid_username("Us er") == False
Expected Output #1:
• Username validation logic successfully passing all AI- generated test cases.

Prompt:-
#Write a Python program to create a function called is_valid_username(username).The function should check whether a username is valid or not.
#The username must Be between 5 and 15 characters long,Contain only letters and numbers,Not start with a number,Not contain any spaces
#After writing the function, create at least 3 different assert test cases to check both valid and invalid usernames.

Code:-

```
assignment8.py > ...
#Task-1
#Write a Python program to create a function called is_valid_username(username).The function should check whether a username is val
#The username must Be between 5 and 15 characters long,Contain only letters and numbers,Not start with a number,Not contain any sp
#After writing the function, create at least 3 different assert test cases to check both valid and invalid usernames.
def is_valid_username(username):
    if len(username) < 5 or len(username) > 15:
        return False
    if not username.isalnum():
        return False
    if username[0].isdigit():
        return False
    if ' ' in username:
        return False
    return True
# Test cases
assert is_valid_username("user123") == True
assert is_valid_username("123user") == False
assert is_valid_username("user name") == False
assert is_valid_username("us") == False
assert is_valid_username("thisisaverylongusername") == False
assert is_valid_username("validUser") == True
print("All test cases passed!")
```

Output:-

```
PS C:\Users\Dell\Desktop\PYTHON>

All test cases passed!
```

Task Description #2 (Even–Odd & Type Classification – Apply AI for Robust Input Handling)
• Task: Use AI to generate at least 3 assert test cases for a function classify_value(x) and implement it using conditional logic and loops.
• Requirements:
o If input is an integer, classify as "Even" or "Odd".
o If input is 0, return "Zero".
o If input is non-numeric, return "Invalid Input".
Example Assert Test Cases:
assert classify_value(8) == "Even"
assert classify_value(7) == "Odd"
assert classify_value("abc") == "Invalid Input"
Expected Output #2:
• Function correctly classifying values and passing all test cases.

Prompt:-
#Write a Python program to create a function named classify_value(x).
#The function should Return "Even" if the number is even,Return "Odd" if the number is odd,Return "Zero" if the input is 0,Return "Invalid Input" if the given value is not a number.
#Write at least 3 assert test cases to verify the function works correctly.

Code:-

```python
#Task-2
#Write a Python program to create a function named classify_value(x).
#The function should Return "Even" if the number is even,Return "Odd" if the
#Write at least 3 assert test cases to verify the function works correctly.
def classify_value(x):
    if not isinstance(x, (int, float)):
        return "Invalid Input"
    if isinstance(x, float) and x % 1 != 0:
        return "Invalid Input"
    if x == 0:
        return "Zero"
    elif x % 2 == 0:
        return "Even"
    else:
        return "Odd"
# Test cases
assert classify_value(4) == "Even"
assert classify_value(7) == "Odd"
assert classify_value(0) == "Zero"
assert classify_value("string") == "Invalid Input"
assert classify_value(3.5) == "Invalid Input"
print("All test cases passed!")
```

Output:-

```
PS C:\Users\Dell\Desktop\PYTHON>

All test cases passed!
```

Task Description #3 (Palindrome Checker – Apply AI for String Normalization)
• Task: Use AI to generate at least 3 assert test cases for a function is_palindrome(text) and implement the function.
• Requirements:
o Ignore case, spaces, and punctuation.
o Handle edge cases such as empty strings and single characters.
Example Assert Test Cases:
assert is_palindrome("Madam") == True
assert is_palindrome("A man a plan a canal Panama") ==True
assert is_palindrome("Python") == False
Expected Output #3:
• Function correctly identifying palindromes and passing all AI-generated tests.

Prompt:-
#Write a Python function called is_palindrome(text) to check whether a given string is a palindrome.
#While checking Ignore capital and small letters,Ignore spaces,Ignore punctuation marks,Handle special cases like empty string and single character.
#Create at least 3 assert test cases to check the function.

Code:-

```python
#Task-3
#Write a Python function called is_palindrome(text) to check whether a given st
#While checking Ignore capital and small letters,Ignore spaces,Ignore punctuati
#Create at least 3 assert test cases to check the function.
import string
def is_palindrome(text):
    # Remove spaces and punctuation, and convert to lowercase
    cleaned_text = ''.join(char for char in text if char.isalnum()).lower()
    return cleaned_text == cleaned_text[::-1]
# Test cases
assert is_palindrome("A man, a plan, a canal, Panama") == True
assert is_palindrome("No 'x' in Nixon") == True
assert is_palindrome("Hello") == False
assert is_palindrome("") == True
assert is_palindrome("a") == True
print("All test cases passed!")
```

Output:-

```
PS C:\Users\Dell\Desktop\P

All test cases passed!
```

Task Description #4 (BankAccount Class – Apply AI for Object-Oriented Test-Driven Development)
• Task: Ask AI to generate at least 3 assert-based test cases for a BankAccount class and then implement the class.
• Methods:
o deposit(amount)
o withdraw(amount)
o get_balance()

Example Assert Test Cases:
acc = BankAccount(1000)
acc.deposit(500)
assert acc.get_balance() == 1500
acc.withdraw(300)
assert acc.get_balance() == 1200
Expected Output #4:
• Fully functional class that passes all AI-generated assertions.

Prompt:-
#Write a Python program to create a class named BankAccount.
#The class should include A method to deposit money,A method to withdraw money,A method to check the current balance Initialize the account with some starting balance.
#Write assert test cases to test deposit, withdraw, and balance checking.

Code:-

```python
#Task-4
#Write a Python program to create a class named BankAccount.
#The class should include A method to deposit money,A method to withdraw m
#Write assert test cases to test deposit, withdraw, and balance checking.
class BankAccount:
    def __init__(self, starting_balance=0):
        self.balance = starting_balance

    def deposit(self, amount):
        if amount > 0:
            self.balance += amount
            return True
        return False

    def withdraw(self, amount):
        if 0 < amount <= self.balance:
            self.balance -= amount
            return True
        return False

    def check_balance(self):
        return self.balance
# Test cases
account = BankAccount(100)
assert account.check_balance() == 100
assert account.deposit(50) == True
assert account.check_balance() == 150
assert account.withdraw(30) == True
assert account.check_balance() == 120
assert account.withdraw(200) == False
print("All test cases passed!")
```

Output:-

```
All test cases passed!
```

Task Description #5 (Email ID Validation – Apply AI for Data Validation)
• Task: Use AI to generate at least 3 assert test cases for a function validate_email(email) and implement the function.
• Requirements:
o Must contain @ and .
o Must not start or end with special characters.
o Should handle invalid formats gracefully.
Example Assert Test Cases:
assert validate_email("user@example.com") == True
assert validate_email("userexample.com") == False
assert validate_email("@gmail.com") == False
Expected Output #5:

• Email validation function passing all AI-generated test cases and handling edge cases correctly.

Prompt:-
#Write a Python function named validate_email(email) to check whether an email address is valid.
#The email should Contain an "@" symbol,Contain a valid domain with ".",Not contain any spaces,Not start or end with special characters.
#Write at least 3 assert test cases to check both valid and invalid email formats.

Code:-

```
#Task-5
#Write a Python function named validate_email(email) to check whether an email add
#The email should Contain an "@" symbol,Contain a valid domain with ".",Not contai
#Write at least 3 assert test cases to check both valid and invalid email formats.
import re
def validate_email(email):
    if ' ' in email:
        return False
    if not re.match(r'^[\w\.-]+@[\w\.-]+\.\w+$', email):
        return False
    return True
# Test cases
assert validate_email("user@example.com") == True
assert validate_email("invalid.email@") == False
assert validate_email("test@domain.co.uk") == True
assert validate_email("user name@example.com") == False
print("All test cases passed!")
```

Output:-

```
All test cases passed!
```